Azure migration notes

> **Status:** background and open items. The concrete production plan — architecture, service tiers, costs and the step-by-step runbook — is production-deployment.md, which chooses **Azure Container Apps**. The repo ships container images (`backend/Dockerfile` , `frontend/Dockerfile` + `nginx.conf.template`) but no infrastructure-as-code or CI/CD yet.

What is already Azure-shaped

Area	State
File storage	Already Azure Blob Storage (`AzureBlobStorageService`), private container, SAS URLs, write-once uploads. Behind a `StorageService` interface. Nothing to migrate
OCR	Already Azure Document Intelligence (`prebuilt-layout`)
LLM	Already Azure OpenAI (text + vision) behind `LLMService` ; accepts a classic resource endpoint or an AI Foundry project URL
Configuration	All secrets, endpoints and thresholds come from environment variables (`core/config.py`); no hardcoded local paths. The same image can run anywhere given the right env
Statelessness	The API and the workers hold no local state. Files go to Blob, everything else to Postgres/Redis. Rendering is done in memory, so no local scratch disk is required
Schema management	Alembic migrations apply cleanly to an empty database (verified), so a new environment is bootstrapped by `alembic upgrade head`
Managed PostgreSQL	No role needs SUPERUSER or BYPASSRLS; the migration owner needs only CREATEROLE — which the Flexible Server admin has. The whole chain was verified as a non-superuser owner
Connection pooling	Tenant context is transaction-local, so the built-in PgBouncer (transaction mode) of Flexible Server works as-is
Images	`backend/Dockerfile` (API, every worker, beat, migrations — the command differs) and `frontend/Dockerfile` (nginx serving the SPA and proxying `/api`)

What is still local-only (and what to do about it)

Component today	Local form	Azure target (proposed)	Notes
PostgreSQL	Docker `postgres:16-alpine` + PgBouncer container	**Azure Database for PostgreSQL – Flexible Server** (v16) with the **built-in PgBouncer** (General Purpose tier)	`DATABASE_URL` with `?sslmode=require` ; private access; PITR backups. Run migrations as a one-off job per release. Set `SEED_ADMIN_PASSWORD` (the seed resets it)
Redis (broker, fair-share counters, rate limiter)	Docker `redis:7-alpine` , no auth	**Azure Managed Redis**, *Non-Clustered* policy	Azure Cache for Redis no longer accepts new customers and retires 2028-09-30. Use `rediss://…:10000/0` for **both** Celery URLs (database 0 only)

Component today	Local form	Azure target (proposed)	Notes
FastAPI app	`uvicorn --reload` on the host	Container App with internal ingress (`backend/Dockerfile`)	`uvicorn … --workers 2`; not `start-api.sh` (it migrates and re-seeds on every start)
Celery workers	one local worker	Three Container Apps (`start-worker.sh extraction	vision
Frontend	Vite dev server + `/api` proxy	Container App `fddt-web` (`frontend/Dockerfile`: nginx serves the build and proxies `/api/*` to the API's internal FQDN)	One origin, so no CORS. nginx caps request bodies (`UPLOAD_BODY_LIMIT`, default 11m; `BULK_UPLOAD_BODY_LIMIT` for bulk zips, default 301m)
Secrets	`backend/.env` file	**Key Vault** referenced by the Container Apps	`JWT_SECRET_KEY`, owner / `fddt_app` / `fddt_platform` passwords, storage connection string, Document Intelligence and OpenAI keys, Redis key, seed password
Azure credentials	Long-lived keys / connection string	**Managed identity** + RBAC (Storage Blob Data Contributor, Cognitive Services User)	The code authenticates with keys/connection strings only; switching to `DefaultAzureCredential` is a code change in `storage_service.py`, `ocr_service.py`, `llm_service.py` (each already sits behind an interface). Note the SAS generation currently needs an account key — with managed identity use *user delegation* SAS
Health/readiness	`GET /health` (does not touch dependencies)	Probe target	Consider a readiness probe that checks Postgres and Redis
Auth	Email/password JWT, in `localStorage`	Same for v1; **Entra ID SSO** is a possible later upgrade	Not started
Email/notifications	Not built	Azure Communication Services (planned)	`EMAIL_*` settings are declared but unused
CI/CD	None	GitHub Actions or Azure DevOps	Suggested stages: lint (`oxlint`), `tsc -b`, `pytest`, build images, migrate, deploy

Things that must change before any production exposure

These are pre-deployment blockers found while documenting, not deployment tasks per se:

1. ~~**Authorization gaps** on upload, file-url and the signature endpoints~~ — **fixed** (details). Still worth a security review of the remaining routes before exposure.
2. ~~**Remove** `/__debug_new_case`~~ — **done** (the public route outside `ProtectedRoute` no longer exists).
3. **TLS + single origin.** Terminate TLS at the Container Apps ingress (or Front Door) and keep the SPA and API on one origin (the nginx proxy) — the backend has no CORS middleware. Blob Storage needs a CORS rule for the app origin (the PDF viewer reads SAS URLs directly).
4. **JWT hardening.** Change `JWT_SECRET_KEY`; consider shorter lifetimes, refresh tokens, and server-side revocation (there is no logout or password reset endpoint). Tokens in `localStorage` are exposed to XSS.

5. **Login abuse.** No rate limiting or lockout on `POST /auth/login`; no password policy beyond "8+ characters" on admin-created accounts.
6. **Issuer registry.** Each company's registry starts empty (the migrated "Default Company" has fake test issuers); load the client's real data, or most documents will fail issuer verification.
7. **Upload limits.** PDF only (by header bytes), per-company size limits (10 MB / 300 MB zip by default, set by a platform admin), corruption and password-protection checks exist (`services/upload_validation.py`, `services/upload_limits.py`). The nginx proxy's body caps (`UPLOAD_BODY_LIMIT` / `BULK_UPLOAD_BODY_LIMIT`) must cover the largest limit any company is given. Still missing: a page-count limit, a malware scan (e.g. Defender for Storage malware scanning) and a Blob lifecycle/quarantine policy.
8. **Stuck-work recovery.** If Redis is down at upload time the document stays `pending` forever; there is no sweeper or retry. Add a periodic reconciliation task.
9. **Observability.** Python `logging`, the audit log, the queue monitor and the per-minute `queue_metrics` / `queue_alert` log lines exist. Ship container logs to Log Analytics and create the alerts in production-deployment.md; Application Insights/OpenTelemetry tracing is not wired in.

Scaling and cost considerations

- **Azure OpenAI is the throughput limit; Document Intelligence is the larger cost.** Per document: one extraction call, an optional issuer-match call, **two vision calls per PDF page** (visual review), plus signature detection and, on demand, comparison calls — about 1 cent of gpt-4.1-mini per 2-page document, against 2 cents of Document Intelligence. Full model: production-deployment.md › Costs. Set the TPM quota for the volume and the app's caps to 80 % of it.
- **Duplicate detection is a linear scan** of every stored page hash of the company per new page. Fine for thousands of pages; move to an index (BK-tree/LSH) or a pre-filter before millions.
- **Worker memory:** a multi-page PDF is rendered to full-resolution arrays (200 DPI); large documents benefit from a memory-limited, low-concurrency worker rather than many parallel tasks.
- **Report generation is synchronous** in the API request; if reports get heavy, move `generate_case_report` into a Celery task and poll (the function is already the unit to move).
- **Pipeline tasks are not retried automatically.** A transient Azure failure marks the document/check `failed`; there is no retry policy or dead-letter handling. Consider Celery `autoretry_for` with backoff on the LLM/OCR calls.
- **Data retention and privacy.** Documents and extracted text (including personal data) are stored indefinitely. Define retention, encryption-at-rest (Azure default), region/residency, and who may read the audit log. Azure OpenAI abuse-monitoring/data-handling terms should be reviewed for the client's data classification.

Suggested order

1. Follow production-deployment.md section 4 (network, registry, PostgreSQL, Redis, Storage, AI resources, Key Vault, environment, migration job, apps, domain).
2. Work through the blockers above that apply (login abuse, JWT hardening, stuck-work recovery).
3. Add CI/CD (suggested stages: `oxlint`, `tsc -b`, `pytest` incl. the Postgres RLS suite, build images, run the migration job, roll the apps) and infrastructure-as-code (Bicep/Terraform) for the resources.
4. Load real issuer data, tune risk weights on real cases, and validate the weak signals (ELA, signature and AI-generation checks) against known-genuine and known-forged documents before relying on them.